Spec 004 — Shareable Room URL + Mid-Game Seat Takeover + Spectator Mode

Status: Draft **Author:** Claude **Date:** 2026-04-19 **Tracker:** github.com/Kitchen-Table-Games/kitchen-table-games/issues/1 **Related:** Spec 001 (Multi-game platform), Spec 002 (Lobby & matchmaking), Spec 003 (Bot seat protocol)

1. Problem

Today games are invite-only: the host creates a room, shares a link, and everyone joins at the start. Three gaps:

- **Late joiners can't join.** Once a game starts, the lobby closes. Friends who arrive 10 minutes in are locked out.
- **Disconnected players can't return.** Reconnection works for a few seconds, but if a player drops for 30 seconds (phone lock, network switch), they're permanently gone. Bot takeover (Spec 003) helps, but there's no way for a human to reclaim their seat.
- **Spectators don't exist.** Friends who want to watch a game have no way to join as observers. Streaming a game requires screen sharing.

Spec 002 (Lobby & matchmaking) addresses discovery and matchmaking. This spec addresses what happens after players are in a game.

2. Proposed Changes — Slow coordinated phases

2.1 Persistent room URL

The URL `/play/{roomCode}` stays valid for the entire game. Currently it redirects to `/lobby` if the game hasn't started. Instead, it should always load the game state.

- Expired rooms (> 24 hours) return a "room expired" message.
- Invalid room codes return a "room not found" message.

2.2 Mid-game seat takeover

When a disconnected player's seat is open (bot-controlled or waiting), the reconnect flow becomes a claim/takeover:

- Player visits `/play/{roomCode}` and sees a lobby-like screen showing which seats are occupied, which are open, and which are bot-controlled.
- Tap an open seat to claim it. Server validates the room is not full, then assigns the player to that seat.
- If the seat is bot-controlled, the bot's state (hand, score, position) transfers to the new player.
- Other players see a notification: "{name} joined as {seat}".

2.3 Spectator mode

Friends who want to watch can join as observers. Spectators see the game state in real time but can't interact.

- Spectators receive a filtered game state (no hidden information — no other players' hands, no deck order).
- Spectator count visible to players: "3 spectators watching".
- Spectators can request to join a seat when one opens.

3. Proposed Design — Slow coordinated phases

3.1 Persistent room URL

The URL `/play/{roomCode}` stays valid for the entire game. Currently it redirects to `/lobby` if the game hasn't started. Instead, it should always load the game state.

- Expired rooms (> 24 hours) return a "room expired" message.
- Invalid room codes return a "room not found" message.

3.2 Mid-game seat takeover

When a disconnected player's seat is open (bot-controlled or waiting), the reconnect flow becomes a claim/takeover:

- Player visits `/play/{roomCode}` and sees a lobby-like screen showing which seats are occupied, which are open, and which are bot-controlled.
- Tap an open seat to claim it. Server validates the room is not full, then assigns the player to that seat.
- If the seat is bot-controlled, the bot's state (hand, score, position) transfers to the new player.
- Other players see a notification: "{name} joined as {seat}".

3.3 Spectator mode

Friends who want to watch can join as observers. Spectators see the game state in real time but can't interact.

- Spectators receive a filtered game state (no hidden information — no other players' hands, no deck order).
- Spectator count visible to players: "3 spectators watching".
- Spectators can request to join a seat when one opens.

4. Client Changes

4.1 Lobby screen

Shows room code, player slots (filled / open / bot), and a "Waiting for players..." state. Host sees a "Start game" button when enough players are seated.

4.2 Game screen

Renders the game state received from the server. Adapts to any game via the game-type registry. Shows a "Bot playing" indicator when a seat is bot-controlled.

4.3 Takeover UX

When a seat is open, a player who visits the room URL sees a simple "Join as {seat}" button. Once seated, they see the game as any other player.

4.4 Spectator UX

Read-only game view. Shows all public information. Displays spectator count. Shows "Waiting for a player to join" when a seat is open.

5. Client Changes

5.1 Lobby screen

Shows room code, player slots (filled / open / bot), and a "Waiting for players..." state. Host sees a "Start game" button when enough players are seated.

5.2 Game screen

Renders the game state received from the server. Adapts to any game via the game-type registry. Shows a "Bot playing" indicator when a seat is bot-controlled.

5.3 Takeover UX

When a seat is open, a player who visits the room URL sees a simple "Join as {seat}" button. Once seated, they see the game as any other player.

5.4 Spectator UX

Read-only game view. Shows all public information. Displays spectator count. Shows "Waiting for a player to join" when a seat is open.

6. Server Changes

6.1 Room lifecycle

Rooms persist for 24 hours after creation. Cleanup runs every 5 minutes.

6.2 Seat management

```
seat_assignments: {
  "0": { type: "human", name: "Alice", connected: true },
  "1": { type: "bot", name: "Bot 1", bot_id: "abc" },
  "2": { type: "human", name: "Bob", connected: false,
         disconnect_at: 1713...},
  "3": { type: "bot", name: "Bot 2", bot_id: "def" }
}
```

6.3 Takeover flow

```
POST /api/rooms/{roomCode}/claim { seat: 2 }
  → validate: seat exists, seat is bot or disconnected, room not full
  → transfer: bot state → new human
  → broadcast: "{name} joined as {seat}"
  → return: { gameToken, gameState }
```

6.4 Spectator API

```
GET /api/rooms/{roomCode}/spectate
  → return: { gameState (filtered), spectatorCount }
```

7. Bot Integration

Bot seats are filled via the bot protocol (Spec 003). Takeover transfers the bot's game state to the new human player.

8. Implementation Phases

1. Persistent room URL — remove redirect, load game state
2. Seat management — track seat types, connections
3. Takeover flow — claim open seat, state transfer
4. Spectator API — filtered state, spectator count
5. Client — lobby screen, game screen, takeover UX, spectator UX

9. Open Questions

- Should takeover require host approval?
- Max spectators?
- Should spectators see bot thinking indicators?
- Should we record spectator sessions for replay?

10. Related

- Spec 001 — Multi-game platform
- Spec 002 — Lobby & matchmaking
- Spec 003 — Bot seat protocol

11. Revision History

- 2026-04-19 — Initial draft